# CSA1709: Artificial Intelligence - SLOT: B

## Lab Experiments

Name: Abdul Razak Shaik
Register No.: 192524358

### Experiment 17: Prolog Program to Sum the Integers from 1 to N

Aim: To write a Prolog program to find the sum of integers from 1 to N.

Algorithm:

1. Start the program.
2. Define a predicate sum(N, S) to calculate the sum of integers from 1 to N.
3. If N = 0, set the sum as 0.
4. Otherwise, recursively calculate the sum up to N-1.
5. Add N to the previous sum.
6. Display the result.
7. Stop.

Program:

```
sum(0, 0).
sum(N, S) :-
   N > 0,
   N1 is N - 1,
   sum(N1, S1),
   S is N + S1.
```

Query:

```
?- sum(10, S).
```

Output:


Result: The sum of integers from 1 to 10 is 55.

## Experiment 18. Prolog Program for a Database with Name and DOB

Aim: To write a Prolog program to create a database containing Name and Date of Birth (DOB) and retrieve the DOB of a person.

Algorithm:

1. Start the program.
2. Define facts containing the person's name and date of birth.
3. Define a predicate dob(Name, Date) to retrieve the DOB.
4. Query the database using a person's name.
5. Display the corresponding DOB.
6. Stop.

Program:

```
dob(john, date(1990, 5, 1)).
dob(jane, date(1985, 12, 10)).
dob(bob, date(1978, 2, 28)).
dob(sue, date(1995, 8, 15)).
dob(tom, date(2000, 4, 22)).
lookup(Name, DOB) :-
    dob(Name, DOB).
```

Query:

?- lookup(john, DOB).

Output:


Result: The DOB of the specified person is successfully retrieved from the database.

## Experiment 19. Prolog Program for Student-Teacher-Sub-Code

Aim: To write a Prolog program to represent the relationship between students, teachers, and subjects/codes and retrieve the corresponding information.

Algorithm:

1. Start the program.
2. Define facts for students and their subjects.
3. Define facts for teachers and the subjects they handle.
4. Define subject codes.
5. Create a predicate to relate student, teacher, and subject code.
6. Query the database.
7. Display the result.
8. Stop.

Program:

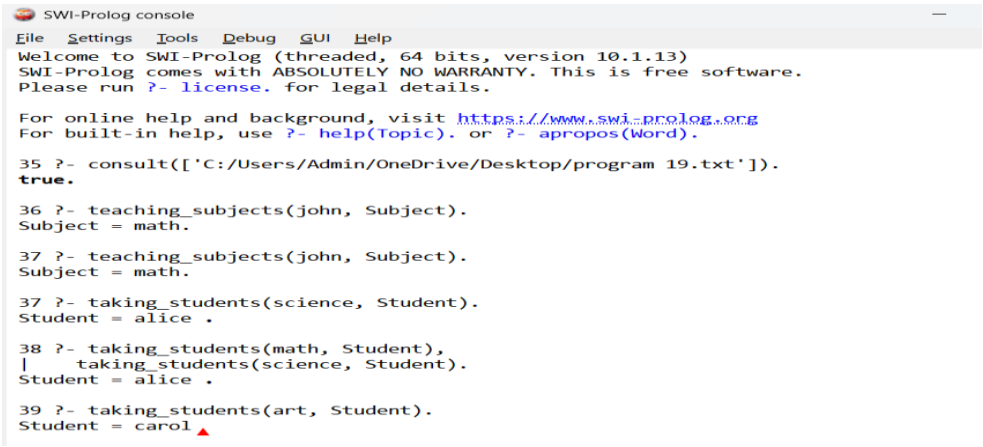
teaches(john, math). teaches(jane, english).
teaches(bob, science). teaches(sue, history).
teaches(tom, art).
takes(alice, math).
takes(alice, science).

```
takes(bob, english).
takes(bob, science).
takes(carol, history).
takes(carol, art).
takes(dave, math).
takes(dave, english).
takes(dave, art).
teaching_subjects(Teacher, Subject) :-
        teaches(Teacher, Subject).
taking_students(Subject, Student) :-
        takes(Student, Subject).
```

Query:

```
?- teaching_subjects(john, Subject).
?- taking_students(science, Student).
?- taking_students(math, Student), taking_students(art, Student).
```

Output




Result: The student, teacher, and corresponding subject code are successfully obtained using Prolog relationships

## Experiment 20. Prolog Program for Planets Database

Aim: To write a Prolog program to create a database of planets and retrieve information about them.

Algorithm:

1. Start the program.
2. Define facts containing the names of planets.
3. Store information such as order, type, and number of moons.
4. Define predicates to retrieve planet information.
5. Query the database.
6. Display the result.
7. Stop.

Program:

```
planet(mercury, 1, terrestrial, 0).
planet(venus, 2, terrestrial, 0).
```

```
planet(earth, 3, terrestrial, 1).
planet(mars, 4, terrestrial, 2).
planet(jupiter, 5, gas_giant, 95).
planet(saturn, 6, gas_giant, 146).
planet(uranus, 7, ice_giant, 28).
planet(neptune, 8, ice_giant, 16).
planet_info(Name, Order, Type, Moons) :-
    planet(Name, Order, Type, Moons).
```

Query:

```
?- planet_info(earth, Order, Type, Moons).
?- planet_info(jupiter, Order, Type, Moons).
```

Output:

```
40 ?- consult(['C:/Users/Admin/OneDrive/Desktop/Program 20.pl']).
true.

41 ?- planet_info(earth, Order, Type, Moons).
Order = 3,
Type = terrestrial,
Moons = 1.

42 ?- planet_info(jupiter, Order, Type, Moons).
Order = 5,
Type = gas_giant,
Moons = 95.

43 ?-
```

Result: The planet database is successfully created and the required information is retrieved using Prolog.

**Experiment 21. Prolog Program to Implement Towers of Hanoi**

Aim

To write a Prolog program to implement the Towers of Hanoi problem using recursion.

Algorithm

1. Start with N disks on the source peg.
2. If there is only one disk, move it directly from source to destination.
3. Otherwise:
    - Move N-1 disks from source to auxiliary peg.
    - Move the largest disk from source to destination.
    - Move N-1 disks from auxiliary peg to destination.
4. Display all the required moves.
5. Stop.

Program

```
hanoi(1, Source, Destination, _) :-

    write('Move disk from '),

    write(Source),

    write(' to '),

    write(Destination),
```

```
    nl.
hanoi(N, Source, Destination, Auxiliary) :-
    N > 1,
    N1 is N - 1,
    hanoi(N1, Source, Auxiliary, Destination),
    write('Move disk from '),
    write(Source),
    write(' to '),
    write(Destination),
    nl,
    hanoi(N1, Auxiliary, Destination, Source).
```

Query

?- hanoi(3, source, destination, auxiliary).

Output

```
43 ?- consult(['C:/Users/Admin/OneDrive/Desktop/program 21.pl']).
true.

44 ?- hanoi(3, source, destination, auxiliary).
Move disk from source to destination
Move disk from source to auxiliary
Move disk from destination to auxiliary
Move disk from source to destination
Move disk from auxiliary to source
Move disk from auxiliary to destination
Move disk from source to destination
true
```

Result

Thus, the Towers of Hanoi problem was successfully implemented using Prolog.

## 22. Prolog Program to Check Whether a Particular Bird Can Fly or Not

Aim

To write a Prolog program to determine whether a particular bird can fly or cannot fly using facts and rules.

Algorithm

1. Start the program.
2. Define facts for birds that can fly.
3. Define facts for birds that cannot fly.
4. Define the predicate can_fly(Bird).
5. Query the required bird.
6. Display whether the bird can fly or not.
7. Stop.

Program

```
can_fly(eagle).

can_fly(parrot).

can_fly(pigeon).

can_fly(crow).
```

```
can_fly(sparrow).

cannot_fly(penguin).

cannot_fly(ostrich).

cannot_fly(emu).

bird_can_fly(Bird) :-

    can_fly(Bird).

bird_cannot_fly(Bird) :-

    cannot_fly(Bird).
```

Query 1

```
?- bird_can_fly(eagle).

?- bird_cannot_fly(penguin).

?- bird_can_fly(penguin).
```

Output:

```
45 ?- consult(['C:/Users/Admin/OneDrive/Desktop/program 22.pl']).
true.

46 ?- bird_can_fly(eagle).
true.

47 ?- bird_cannot_fly(penguin).
true.

48 ?- bird_can_fly(penguin).
false.

49 ?-
```

Result

Thus, the Prolog program successfully determines whether a particular bird can fly or not.

**Experiment 23. Prolog Program to Implement Family Tree**

Aim

To write a Prolog program to represent a family tree and determine relationships such as parent, grandparent, sibling, and child.

Algorithm

1. Start the program.
2. Define facts for parent-child relationships.
3. Define the father and mother relationships.
4. Define rules for grandparent.
5. Define rules for sibling.
6. Query the required relationship.
7. Display the result.
8. Stop.

Program

```
father(ramesh, rahul).

father(ramesh, priya).

father(suresh, anjali).
```

```
mother(sita, rahul).

mother(sita, priya).

mother(geetha, anjali).

parent(X, Y) :-

    father(X, Y).

parent(X, Y) :-

    mother(X, Y).

grandparent(X, Z) :-

    parent(X, Y),

    parent(Y, Z).

sibling(X, Y) :-

    parent(P, X),

    parent(P, Y),

    X \= Y.
```

Query

```
?- parent(X, rahul).

?- sibling(rahul, X).

?- grandparent(X, rahul).
```

Output:

```
49 ?- consult(['C:/Users/Admin/OneDrive/Desktop/program 23.pl']).
true.

50 ?- parent(X, rahul).
X = ramesh ;
X = sita.

51 ?- sibling(rahul, X).
X = priya ;
X = priya.

52 ?- grandparent(X, rahul).
false.

53 ?-
```

Result

Thus, the family tree was successfully implemented using Prolog facts and rules.

**Experiment 24. Prolog Program to Suggest Dieting System Based on Disease**

Aim: To write a Prolog program to suggest a suitable dieting system based on a person's disease.

Algorithm

1. Define diseases and recommended foods.
2. Define diseases and foods to avoid.
3. Create a predicate to suggest a diet.
4. Query the disease.
5. Display the recommended foods and foods to avoid.

Program

recommended_diabetes(vegetables).

recommended_diabetes(whole_grains).

recommended_diabetes(fruits_in_moderation).

avoid_diabetes(sugary_foods).

```
avoid_diabetes(soft_drinks).
recommended_hypertension(fruits).
recommended_hypertension(vegetables).
recommended_hypertension(low_salt_food).
avoid_hypertension(salty_foods).
avoid_hypertension(processed_foods).
recommended_anemia(leafy_vegetables).
recommended_anemia(beetroot).
recommended_anemia(iron_rich_foods).
avoid_anemia(excess_tea).
avoid_anemia(excess_coffee).
diet(Disease, Food) :-
    Disease = diabetes,
    recommended_diabetes(Food).
diet(Disease, Food) :-
    Disease = hypertension,
    recommended_hypertension(Food).
diet(Disease, Food) :-
    Disease = anemia,
    recommended_anemia(Food).
```

Query :

```
?- diet(diabetes, Food).
```

?- diet(hypertension, Food).

Output:

```
53 ?- consult(['C:/Users/Admin/OneDrive/Desktop/program 24.pl']).
true.

54 ?- diet(diabetes, Food).
Food = vegetables ;
Food = whole_grains ;
Food = fruits_in_moderation.

55 ?- diet(hypertension, Food).
Food = fruits ;
Food = vegetables ;
Food = low_salt_food.

56 ?-
```

Result: Thus, a Prolog-based diet suggestion system was successfully implemented based on disease.

**Experiment 25. Prolog Program to Implement Monkey Banana Problem**

Aim

To write a Prolog program to implement the Monkey Banana Problem, where a monkey uses a box to reach bananas placed at a height.

Algorithm

1. The monkey is initially on the floor.
2. The box is initially at another location.
3. The monkey moves to the box.
4. The monkey pushes the box below the bananas.
5. The monkey climbs onto the box.
6. The monkey grabs the bananas.
7. Display the sequence of actions.

Program

move(state(middle, onfloor, middle, hasnot),

state(middle, onfloor, middle, hasnot)) :-

```
    write('Monkey is already at the middle.'), nl.
move(state(Pos1, onfloor, Pos1, Has),
     state(Pos2, onfloor, Pos2, Has)) :-
    Pos1 \= Pos2,
    write('Monkey moves from '),
    write(Pos1),
    write(' to '),
    write(Pos2),
    nl.
push(state(Pos, onfloor, Pos, Has),
     state(middle, onfloor, middle, Has)) :-
    Pos = middle,
    write('Monkey pushes the box to the middle.'), nl.
climb(state(middle, onfloor, middle, Has),
      state(middle, onbox, middle, Has)) :-
    write('Monkey climbs onto the box.'), nl.
grab(state(middle, onbox, middle, hasnot),
     state(middle, onbox, middle, has)) :-
    write('Monkey grabs the bananas.'), nl.
solve :-
    write('Monkey Banana Problem'), nl,
    write('Monkey moves to the box.'), nl,
```

```
    write('Monkey pushes the box below the bananas.'), nl,
    write('Monkey climbs onto the box.'), nl,
    write('Monkey grabs the bananas.'), nl.
```

Query : ?- solve.

Output:

```
56 ?- consult(['C:/Users/Admin/OneDrive/Desktop/program 25.pl']).
true.

57 ?- solve.
Monkey Banana Problem
Monkey moves to the box.
Monkey pushes the box below the bananas.
Monkey climbs onto the box.
Monkey grabs the bananas.
true.
```

Result : Thus, the Monkey Banana Problem was successfully implemented in Prolog.

**Experiment 26. Prolog Program for Fruit and Its Color Using Backtracking**

Aim: To write a Prolog program to find the color of different fruits using backtracking.

Algorithm

1. Start the program.
2. Define facts containing fruits and their colors.
3. Use the predicate fruit_color(Fruit, Color).
4. Query for a fruit or color.
5. Prolog uses backtracking to find all matching facts.
6. Display the results.
7. Stop.

Program :

```
fruit_color(apple, red).

fruit_color(apple, green).
```

fruit_color(banana, yellow).

fruit_color(orange, orange).

fruit_color(grapes, green).

fruit_color(grapes, purple).

fruit_color(mango, yellow).

fruit_color(strawberry, red).

Query 1 – Find the color of apple

?- fruit_color(apple, Color).

?- fruit_color(Fruit, red).

Output :

```
58 ?- consult(['C:/Users/Admin/OneDrive/Desktop/program 26.pl']).
true.

59 ?- fruit_color(apple, Color).
Color = red ;
Color = green.

60 ?- fruit_color(Fruit, red).
Fruit = apple ;
Fruit = strawberry.

61 ?-
```

Result:

Thus, the fruit and color database using backtracking was successfully implemented in Prolog.

**Experiment 27. Prolog Program to Implement Best First Search Algorithm**

Aim

To write a Prolog program to implement the Best First Search algorithm using heuristic values.

Algorithm

1. Start from the initial node.
2. Store the nodes along with their heuristic values.
3. Select the node having the lowest heuristic value.
4. Expand the selected node.
5. Repeat the process until the goal node is reached.
6. Display the path from the start node to the goal node.

Program:

```
edge(a,b).

edge(a,c).

edge(b,d).
```

```
edge(b,e).
edge(c,f).
edge(c,g).
edge(e,h).
edge(f,h).
heuristic(a,7).
heuristic(b,6).
heuristic(c,5).
heuristic(d,4).
heuristic(e,3).
heuristic(f,2).
heuristic(g,6).
heuristic(h,0).
best_first(Start,Goal,Path) :-
    best_search(Start,Goal,[Start],RevPath),
    reverse(RevPath,Path).
best_search(Goal,Goal,Visited,Visited).
best_search(Current,Goal,Visited,Path) :-
    findall(H-Next,
            (edge(Current,Next),
             \+ member(Next,Visited),
             heuristic(Next,H)),
```

```
        List),
    List \= [],
    choose_best(List,Next),
    best_search(Next,Goal,[Next|Visited],Path).
choose_best([_-N],N).
choose_best([H1-N1,H2-N2|Rest],Best) :-
    H1 =< H2,
    choose_best([H1-N1|Rest],Best).
choose_best([H1-N1,H2-N2|Rest],Best) :-
    H1 > H2,
    choose_best([H2-N2|Rest],Best).
```

Query:

?- best_first(a, h, Path).

Output

```
62 ?- best_first(a, h, Path).
Path = [a, c, f, h] ;
false.

63 ?- best_first(a, h, Path).
Path = [a, c, f, h]
  Possible actions:
  ; (n,r,space,TAB): redo               | t:            trace&redo
  *:                 show choicepoint   | . (c,a,RET): stop
  w:                 write              | p:            print
  +:                 max_depth*5        | -:            max_depth//5
  b:                 break              | h (?):        help

Action? .
```

Result

Thus, the Best First Search algorithm was successfully implemented using Prolog.

**Experiment 28. Prolog Program for Medical Diagnosis**

Aim

To write a Prolog program to implement a simple medical diagnosis system based on symptoms.

Algorithm

1. Start the program.
2. Define symptoms for different diseases.
3. Define rules connecting symptoms with diseases.
4. Query the symptoms of a patient.
5. Use the rules to identify the possible disease.
6. Display the diagnosis.
7. Stop.

Program

```
symptom(ravi, fever).
```

```
symptom(ravi, cough).
symptom(ravi, body_pain).
symptom(priya, fever).
symptom(priya, sneezing).
symptom(priya, running_nose).
symptom(anu, stomach_pain).
symptom(anu, vomiting).
diagnosis(Person, flu) :-
  symptom(Person, fever),
  symptom(Person, cough),
  symptom(Person, body_pain).
diagnosis(Person, common_cold) :-
  symptom(Person, fever),
  symptom(Person, sneezing),
  symptom(Person, running_nose).
diagnosis(Person, food_poisoning) :-
  symptom(Person, stomach_pain),
  symptom(Person, vomiting).
```

Query 1

```
?- diagnosis(ravi, Disease).
```

Output

```
64 ?- diagnosis(ravi, Disease).
Disease = flu .

65 ?- diagnosis(ravi, Disease).
Disease = flu
  Possible actions:
  ; (n,r,space,TAB): redo                | t:            trace&redo
  *:                 show choicepoint    | . (c,a,RET):  stop
  w:                 write               | p:            print
  +:                 max_depth*5         | -:            max_depth//5
  b:                 break               | h (?):        help

Action?
```

Result

Thus, a simple medical diagnosis system was successfully implemented using Prolog.

**Experiment 29. Prolog Program for Forward Chaining**

Aim

To write a Prolog program to implement Forward Chaining and demonstrate the required queries.

Algorithm

1. Start with known facts.
2. Check the rules whose conditions match the known facts.
3. Apply the matching rules.
4. Add the newly derived facts to the knowledge base.
5. Continue until the required conclusion is obtained or no new fact can be derived.
6. Display the result.

Program

```
fact(sunny).

fact(weekend).

rule(go_out) :-
   fact(sunny),
   fact(weekend).

rule(play_cricket) :-
   fact(go_out).

derive(X) :-
   rule(X),
   assertz(fact(X)),
   write(X),
   write(' is derived.'), nl.
```

Query 1

```
?- rule(go_out).
```

Output

```
65 ?- consult(['C:/Users/Admin/OneDrive/Desktop/program 29.pl']).
true.

66 ?- rule(go_out).
true.
```

Result

Thus, the Forward Chaining concept was successfully implemented in Prolog by deriving new facts from existing facts and rules.

**Experiment 30. Prolog Program for Backward Chaining**

Aim

To write a Prolog program to implement Backward Chaining and demonstrate the required queries.

Algorithm

1. Start with a goal.
2. Check whether the goal is a known fact.
3. If it is not a fact, find a rule whose conclusion matches the goal.
4. Treat the rule's conditions as sub-goals.
5. Recursively prove each sub-goal.
6. If all sub-goals are true, the original goal is true.
7. Display the result.

Program

```
rain.

cloudy.

wet_ground :-

    rain.

carry_umbrella :-

    rain,

    cloudy.

go_out :-

    carry_umbrella.
```

Query 1

```
?- wet_ground.

?- carry_umbrella.

?- go_out.
```

Output

```
67 ?- consult(['C:/Users/Admin/OneDrive/Desktop/program 30.pl']).
true.

68 ?- wet_ground.
true.

69 ?- carry_umbrella.
true.

70 ?- go_out.
true.

71 ?-
```

Result

Thus, the Backward Chaining mechanism was successfully demonstrated by starting from the goal and checking the required conditions.